Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 2

«Последовательный интерфейс UART»
Дисциплина: Проектирование вычислительных систем

Вариант: **1. «Светофор»**

Выполнили студенты группы P3410:
Нестеров Владислав Алексеевич
Джохадзе Анна Бекаевна

Преподаватель:
Пинкевич Василий Юрьевич

Санкт-Петербург, 2026

1. Цель и задание работы

Цель работы — изучить обмен данными по UART, устройство контроллера USART и систему прерываний STM32; разработать драйвер UART с буферизацией для режимов опроса и прерываний.

Для варианта 1 требуется доработать светофор из лабораторной работы № 1: добавить управление обработкой кнопки и длительностью красного через UART, запрос текущего состояния и переключение способа обслуживания UART. Скорость обмена — **57600 бит/с**. Обработка команд должна работать одновременно с переключением светодиодов и опросом кнопки.

В обоих режимах каждый принятый символ возвращается в терминал как эхо. Ответ на команду начинается с новой строки. Команды установки параметров подтверждаются сообщением `ok`, неизвестная команда сопровождается сообщением об ошибке. В режиме опроса прерывания USART должны быть запрещены. Приём в обоих режимах и передача в режиме прерываний не должны ожидать завершения аппаратного обмена.

Программа разработана на C для SDK-1.1M с STM32F427VIT6, без операционной системы реального времени. UART реализован собственным регистровым драйвером. HAL используется для начального запуска, тактирования, получения частоты APB2 и миллисекундного времени; функции передачи и приёма HAL UART и DMA не применяются.

2. Аппаратная конфигурация и параметры обмена

Встроенный USB–UART отладчика соединён с USART6. На компьютере используется виртуальный COM-порт интерфейса 1 отладчика; интерфейс 0 предназначен для отладки. Назначение выводов приведено в таблице 1.

Сигнал	**Вывод**	**Настройка / активный уровень**
USART6_TX	PC6	AF8; push-pull; без подтяжки
USART6_RX	PC7	AF8; внутренняя подтяжка вверх
Кнопка	PC15	Вход GPIO; нажатие — логический 0
Зелёный	PD13	Выход GPIO; включение — логическая 1
Жёлтый	PD14, PD15	PD14 = 1; PD15 = 0
Красный	PD14, PD15	PD14 = 0; PD15 = 1

Таблица 1. Используемые выводы SDK-1.1M

Красный и жёлтый кристаллы образуют двухвыводный светодиод со встречным включением. Модуль стенда исключает одновременный запрос этих цветов и записывает маску PD13–PD15 одной операцией через `BSRR`.

Источник системного тактирования — внутренний HSI 16 МГц, PLL выключен, делители AHB, APB1 и APB2 равны 1. Поэтому частота USART6 на APB2 равна 16 МГц. LSE отключён, поскольку PC15 занят кнопкой. Для временных интервалов используется SysTick с шагом 1 мс.

USART6 работает в асинхронном полнодуплексном режиме: 8 бит данных, без чётности, 1 стоп-бит, без управления потоком (**8N1**). Один кадр содержит стартовый бит, восемь информационных и стоповый бит — всего 10 бит. При 57600 бит/с длительность кадра составляет приблизительно 173,61 мкс; предельная скорость полезных данных — 5760 байт/с в каждом направлении.

При передискретизации ×16 значение `BRR` в программе вычисляется округлением отношения частоты APB2 к заданной скорости:

BRR = round(16 000 000 / 57 600) = 278 = 0x0116.

Мантисса делителя равна 17, дробная часть — 6/16. При номинальной частоте HSI фактическая скорость составляет 16 000 000 / 278 ≈ 57553,96 бит/с, отклонение от заданной — около −0,08 %. Это расчёт по номинальной частоте, без учёта погрешности HSI.

3. Логика светофора и управление параметрами

3.1. Последовательность состояний

Светофор циклически проходит состояния `red`, `green`, `blinking green`, `yellow`. При начальных настройках красный длится 4 с, постоянный зелёный — 1 с, мигающий зелёный — 2 с, жёлтый — 1 с. Мигание содержит восемь полупериодов по 250 мс, то есть четыре включения зелёного. Начальное отношение длительностей красного и постоянного зелёного равно 4:1.

На рисунке 1 величина T означает установленную обычную длительность красного. Запрос от кнопки сокращает ближайший красный до T/4, сохраняя порядок состояний. В состоянии постоянного зелёного нажатие игнорируется. Во время мигания или жёлтого сохраняется признак `shorten_next_red`; при входе в красный этот признак расходуется. При нажатии во время красного срок пересчитывается от начала текущей фазы. Если четверть периода уже прошла, выполняется переход к зелёному после подтверждения нажатия.

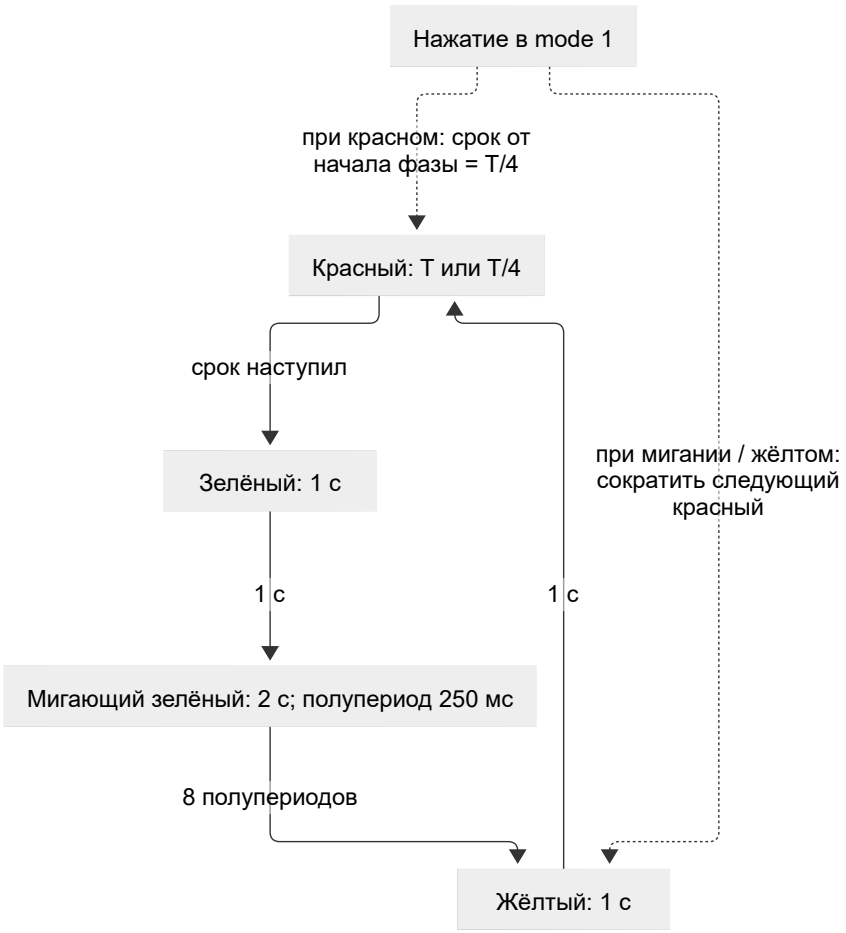


Рис. 1. Автомат светофора и обработка запроса кнопки

3.2. Режим кнопки и изменение тайм-аута

В `mode 1` новые нажатия обрабатываются; в `mode 2` они игнорируются автоматом. Кнопка продолжает опрашиваться в обоих режимах, фильтр дребезга использует интервал стабильности 30 мс. Одно удержание создаёт одно событие. Поэтому включение `mode 1` при уже зажатой кнопке не создаёт нового нажатия: требуется отпустить её и нажать снова.

Ранее принятый запрос при переходе в `mode 2` не отменяется. Команда отключает обработку новых нажатий. Это правило реализации, поскольку задание отдельно не определяет судьбу уже принятого запроса.

Команда `set timeout X` принимает целое число секунд от 1 до 2147483. Верхняя граница обеспечивает интервал меньше `INT32_MAX` миллисекунд для сравнения сроков через `(int32_t)(now_ms - deadline_ms)`. Новый тайм-аут применяется сразу. Если сейчас горит красный, срок вычисляется от начала этого красного с сохранением признака сокращения. Если срок уже прошёл, автомат обновляется в том же вызове.

Команда меняет только красный период. Зелёный, мигание и жёлтый сохраняют свои длительности. Например, при `set timeout 8` обычный красный длится 8 с, сокращённый — 2 с. Настройки хранятся в оперативной памяти; после сброса восстанавливаются `mode 1`, `timeout 4`, режим `I`.

4. Архитектура программы

4.1. Компоненты и их связи

Связи модулей показаны на рисунке 2 назначение файлов приведено в таблице 2. Номера выводов скрыты в аппаратных модулях. Консоль работает через интерфейс драйвера UART, а автомат светофора передаёт логические значения цветов функции обратного вызова `TrafficLightOutput`.

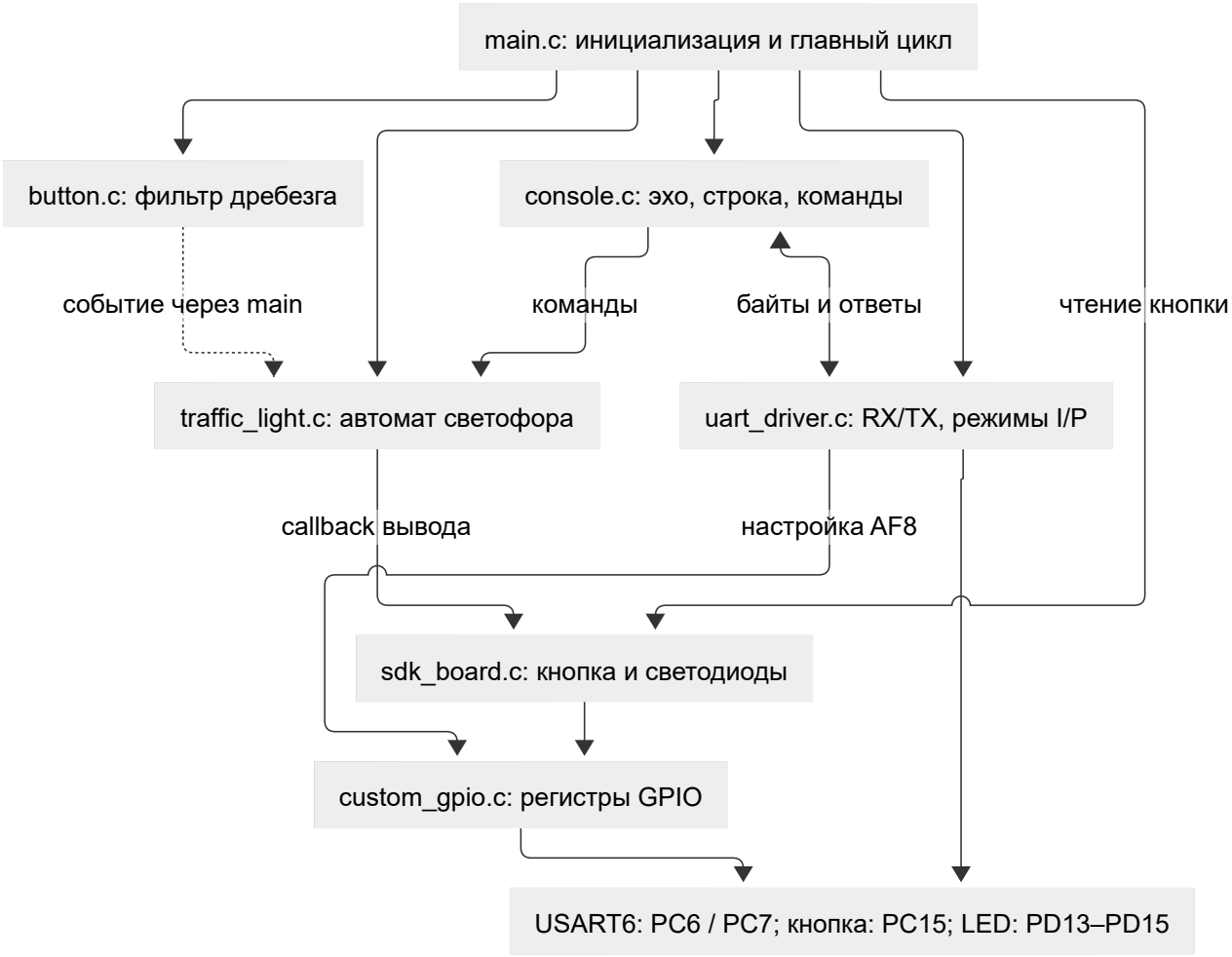


Рис. 2. Архитектура программы и аппаратные зависимости

Модуль	**Ответственность**
`main.c`	Инициализация и последовательный вызов обработчиков
`uart_driver.c/.h`	USART6, кольцевые RX/TX-буферы, опрос, IRQ и смена I/P
`console.c/.h`	Эхо, сбор строки, проверка параметров, ответы и выполнение команд
`traffic_light.c/.h`	Автомат светофора, разрешение кнопки, изменение красного периода
`button.c/.h`	Фильтр дребезга и однократные события нажатия/отпускания
`sdk_board.c/.h`	Физические выводы кнопки и светодиодов, преобразование масок
`custom_gpio.c/.h`	Регистровый GPIO-драйвер; настройка AF8 для USART6

Таблица 2. Программные модули и их назначение

4.2. Главный цикл

После начальной настройки запускаются драйвер UART и консоль. Главный цикл сначала вызывает `uart_poll()`, затем один раз получает время, обновляет автомат светофора, фильтр кнопки, обрабатывает событие нажатия и вызывает консоль. Порядок соответствует текущему `main.c` и показан на рисунке 3.

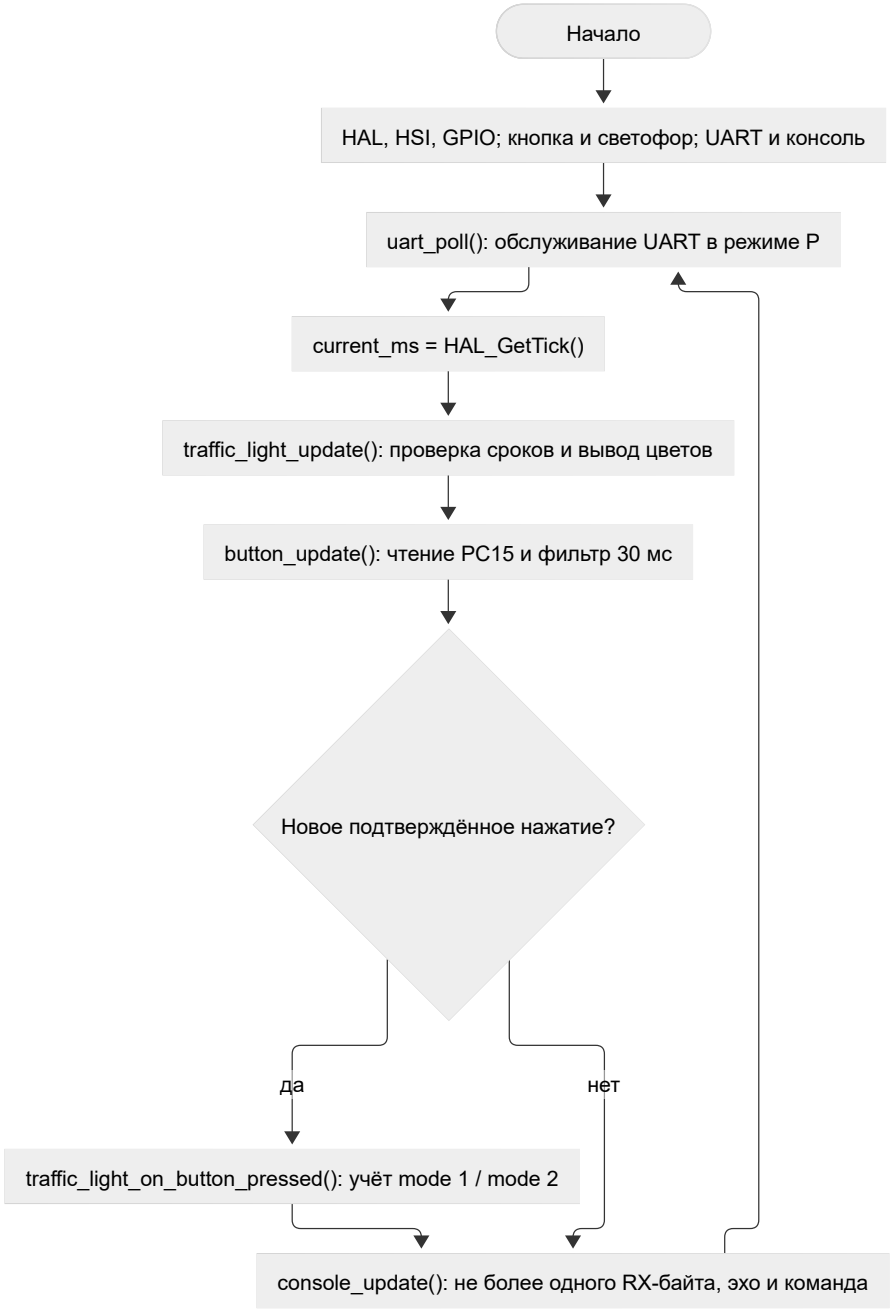


Рис. 3. Блок-схема прикладного алгоритма

В режиме `I` функция `uart_poll()` сразу возвращает управление: USART6 обслуживается обработчиком прерывания. В режиме `P` за вызов выполняется одна проверка аппаратных флагов. `console_update()` извлекает не более одного байта за итерацию. При отсутствии данных или свободного места для полного ответа функция возвращается без ожидания.

Светофор проверяет сроки по `HAL_GetTick()`; `HAL_Delay()` и ожидание ввода не используются. Обновление автомата ограничено 16 шагами за вызов. Сроки штатных переходов отсчитываются от запланированного времени предыдущего перехода, что уменьшает накопление задержек главного цикла.

5. Реализация драйвера UART

5.1. Настройка контроллера и регистры

Драйвер включает тактирование и сбрасывает USART6 через RCC, очищает очереди, задаёт BRR, включает приёмник, передатчик и USART. PC6 и PC7 настраиваются через собственный GPIO-драйвер на AF8. Затем задаётся приоритет USART6, равный 5, очищается ожидающее прерывание и включается режим I.

Используемые регистры и флаги перечислены в таблице 3. Параметры 8N1 и передискретизация ×16 выбираются нулевыми значениями соответствующих полей после сброса.

Регистр	**Использование в драйвере**
RCC_APB2ENR, RCC_APB2RSTR	Тактирование и начальный сброс USART6
GPIOC_MODER, AFRL	Альтернативный режим PC6/PC7, функция AF8
USART_BRR	Делитель скорости обмена
USART_CR1	UE, TE, RE; разрешения RXNEIE, TXEIE; отключение прочих источников при смене режима
USART_CR2, CR3	Один стоп-бит, без flow control и DMA; EIE для ошибок приёма в режиме I
USART_SR	RXNE — байт принят; TXE — DR доступен для следующего байта; ORE, NE, FE, PE — ошибки
USART_DR	Чтение принятого и запись передаваемого байта
NVIC	Приоритет 5, разрешение/запрет и очистка pending USART6

Таблица 3. Регистры и аппаратные признаки, используемые драйвером

5.2. Общая функция обслуживания и два режима

Два варианта работы реализованы в одном модуле с общей функцией service_hardware(), поэтому обработка флагов и очередей не дублируется. Схема приведена на рисунке 4.

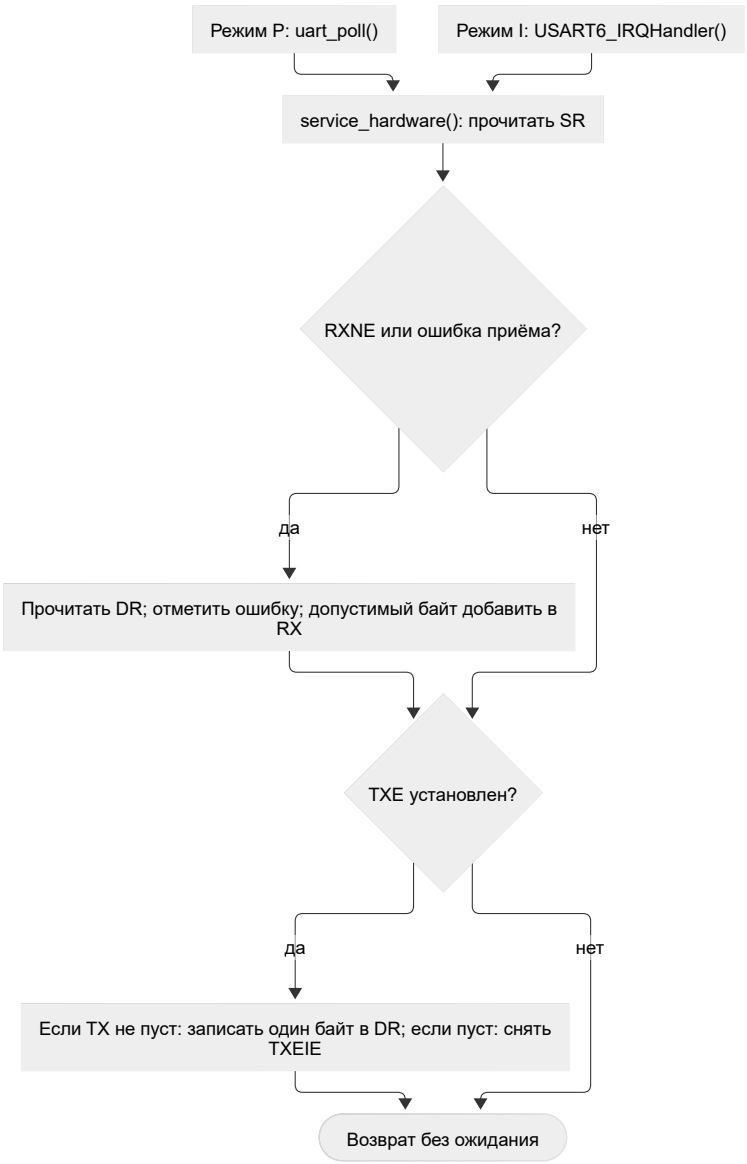


Рис. 4. Обслуживание аппаратуры при опросе и в прерывании

В **режиме опроса P** `uart_poll()` вызывает `service_hardware()` из главного цикла. Линия USART6 в NVIC и источники `RXNEIE`, `TXEIE`, `PEIE`, `TCIE`, `EIE` выключены. При отсутствии принятого байта `uart_read_byte()` возвращает `false`. Передача также буферизована и не содержит ожидания флага `TXE`.

В **режиме прерываний I** включаются `RXNEIE`, `EIE` и линия USART6 в NVIC. При наличии данных в TX разрешается `TXEIE`. Обработчик `USART6_IRQHandler()` только вызывает обслуживание аппаратуры; разбор строк, команды и светофор выполняются в главном цикле. При опустошении TX-очереди `TXEIE` выключается, чтобы не создавать лишних прерываний.

Флаг `TXE` означает возможность записать следующий байт в регистр данных, а не завершение передачи последнего стоп-бита на линии. Возврат `uart_write()` подтверждает помещение сообщения в очередь. Он не означает, что сообщение уже передано компьютеру.

5.3. Буферизация и неблокирующий интерфейс

RX-буфер содержит 512 байт, TX-буфер — 2048 байт. Одна ячейка каждого кольца остаётся свободной: полезные ёмкости равны 511 и 2047 байт. При равенстве `head` и `tail` очередь пуста; при совпадении следующего `head` с `tail` она

полна. Переход индекса реализован маской `size - 1`, а степень двойки проверена `_Static_assert`.

Для TX обработчик только потребляет данные, а единственный производитель — главный цикл. `uart_write()` сначала проверяет свободное место, копирует сообщение с разрешёнными прерываниями и затем публикует новый `tx_head` в короткой критической секции. При недостатке места сообщение не добавляется, возвращается `false`; частичной записи нет. Барьер `__DMB()` обеспечивает публикацию индекса после записи содержимого.

Критические секции сохраняют прежний `PRIMASK` и восстанавливают его при выходе. Они применяются при извлечении RX-байта, публикации TX-индекса и переключении режима. Длительных ожиданий внутри них нет. В режиме P краткая секция защищает один проход обслуживания, при этом SysTick в целом остаётся включённым и обеспечивает время светофора.

Сводка публичных функций приведена в таблице 4.

Функция	**Контракт**
`uart_init()`	Настройка USART6, GPIO и очередей; начальный режим I
`uart_poll()`	Один проход обслуживания в P; немедленный возврат в I
`uart_read_byte()`	Извлечь один байт либо вернуть false
`uart_write()`	Добавить сообщение целиком либо вернуть false
`uart_tx_free()`	Количество свободных ячеек TX
`uart_set_interrupts()`	Переключить I/P без очистки RX/TX
`uart_interrupts_enabled()`	Вернуть текущий режим обслуживания
`uart_take_rx_error()`	Прочитать и сбросить программный признак ошибки RX

Таблица 4. Интерфейс собственного драйвера UART

5.4. Переключение режима и обработка ошибок

`uart_set_interrupts()` сохраняет `PRIMASK`, запрещает линию USART6 и её источники, один раз обслуживает аппаратные флаги, меняет `use_interrupts` и очищает pending в NVIC. Для режима I затем включаются нужные источники и линия; для P они остаются запрещёнными. Содержимое очередей, индексы и настройки USART не сбрасываются. Благодаря этому команда переключения и её ответ продолжают передаваться через тот же канал.

При установленном `RXNE` или признаках ошибки драйвер читает сначала `SR`, затем `DR`. Сохраняется программный признак ошибки. Байт с `NE`, `FE` или `PE` не добавляется в RX; переполнение программного кольца также отмечается как ошибка. При `ORE` байты могли быть утрачены, поэтому текущую команду требуется считать повреждённой.

Консоль при обнаружении ошибки переходит в режим отбрасывания строки до её завершения и выводит `ERROR`. Для ограничения нагрузки консоль не извлекает RX, если в TX осталось меньше 256 свободных байт: этот запас предназначен для эха и полного ответа. Очереди конечны, поэтому для штатной проверки следующую команду отправляют после ответа на предыдущую. Устойчивость восстановления при произвольном потоке, включая несколько команд при переполнении, требует отдельной аппаратной проверки.

6. Командный интерфейс

6.1. Формат ввода и ответы

Команда набирается латинскими строчными буквами и завершается CR, LF или CRLF. Пара CRLF считается одним окончанием строки. Пробелы и табуляции с краёв удаляются; внутри команды ожидается точное написание с одним пробелом. Пустая строка не выполняет действие и не формирует отдельного ответа.

Буфер строки содержит 96 байт: до 95 символов и завершающий ноль. Поддерживаются Backspace и DEL. Слишком длинная строка и недопустимые символы переводят консоль в режим отбрасывания до Enter, после которого выдаётся ошибка. Эхо ставится в очередь до обработки символа. Ответ формируется последовательностью CRLF, текст, CRLF.

Команды и их действие перечислены в таблице 5. Поле `timeout` в статусе обозначает настроенную обычную длительность красного в секундах; оно не является обратным отсчётом. Во время погашенного полупериода мигания статус остаётся `blinking green`.

Команда	**Действие и ответ**
`?`	Текущая фаза; mode 1/2; timeout X; I/P. Например: `red; mode 1; timeout 4; I`
`set mode 1`	Разрешить новые нажатия; `OK`
`set mode 2`	Игнорировать новые нажатия; `OK`
`set timeout X`	Изменить красный период; `OK` при допустимом X
`set interrupts on`	Перейти в I; `OK`
`set interrupts off`	Перейти в P; `OK`
Неизвестная команда	`ERROR: unknown command`
Неверный тайм-аут	`ERROR: timeout must be an integer from 1 to 2147483 seconds`

Таблица 5. Поддерживаемые команды варианта 1

При проверке тайм-аута принимаются только десятичные цифры и положительное значение в установленном диапазоне. До умножения накопленного числа на 10 проверяется верхняя граница. Поэтому отрицательные, дробные, пустые и слишком большие значения отклоняются без изменения прежней настройки.

6.2. Пример ожидаемого диалога

Ниже приведён пример ожидаемого обмена при проверке команд. Цвет в ответе `?` зависит от момента запроса; в примере выбран `red`. Это иллюстрация протокола, а не сохранённый журнал COM-порта. Строки с командами видимы благодаря эху стенда.

```
?
red; mode 1; timeout 4; I
set mode 2
OK
set timeout 8
OK
set interrupts off
OK
?
red; mode 2; timeout 8; P
set interrupts on
OK
set mode 1
OK
set timeout 4
OK
unknown
ERROR: unknown command
```

7. Сборка, запуск и проверка

7.1. Порядок запуска на стенде

Файлы `Core/Inc` и `Core/Src` лабораторной работы № 2 переносятся в существующий проект STM32CubeIDE `light` для STM32F427VIT6 с заменой одноимённых прикладных файлов. Системные файлы, startup, HAL и скрипт компоновщика сохраняются. После обновления дерева проекта выполняются Clean и Build. Обработчик `USART6_IRQHandler()` уже определён в `uart_driver.c`; второго определения в проекте быть не должно.

Настройка USART6 выполняется программно, дополнительная генерация CubeMX не требуется. Файл прошивки загружается через DBG USB с действующей конфигурацией OpenOCD. Для проверки программа должна непрерывно выполняться: остановка на breakpoint нарушает обслуживание UART и временные интервалы.

В терминале выбирается COM-порт интерфейса 1 и параметры 57600, 8N1, flow control None. Local echo отключается, поскольку символы возвращает стенд. Если терминал открыт после запуска, стартовое сообщение могло быть пропущено; текущие настройки запрашиваются командой `?` и Enter.

7.2. Результаты анализа реализации

Проверка исходных файлов подтверждает наличие всех команд варианта 1, скорости 57600 бит/с, двух способов обслуживания USART6, кольцевых RX/TX-очередей и неблокирующих контрактов приёма и передачи. Порядок вызовов в главном цикле сохраняет совместную работу светофора, кнопки и консоли. В режиме P код явно запрещает прерывания соответствующего USART; системный SysTick продолжает использоваться для времени.

В README комплекта зафиксированы ранее выполненные компиляция и компоновка для Cortex-M4, а также отдельная проверка логики команд и светофора. Сохранённого журнала сборки и протокола UART в составе лабораторной нет. Поэтому это сведение о предыдущей проверке, а не результат нового запуска прошивки при подготовке отчёта.

7.3. Контрольные сценарии на SDK-1.1M

В таблице 6 приведены действия и ожидаемые результаты аппаратной проверки. Численные интервалы указаны как параметры алгоритма, а не как измеренные значения. Результаты обмена через COM-порт и измерения времён в предоставленных материалах не зафиксированы.

№	Действия	Ожидаемый результат
1	Сброс; запрос `?`	Текущий цвет, mode 1, timeout 4, I; символы ввода возвращаются эхом
2	Наблюдать цикл без кнопки	Красный 4 с, зелёный 1 с, мигание 2 с, жёлтый 1 с
3	Нажать в начале красного; затем после 1 с красного	В первом случае красный заканчивается на 1 с от начала; во втором — переход после подтверждения нажатия
4	Нажать во время мигания или жёлтого	Порядок фаз сохраняется; ближайший красный 1 с, следующий без запроса — 4 с
5	`set mode 2`; нажать кнопку; затем `set mode 1` и нажать заново	В mode 2 новые нажатия игнорируются; после включения обработки новое нажатие сокращает красный
6	`set timeout 8`; запросить статус и проверить кнопку	`OK`; timeout 8; красный 8 с без запроса и 2 с с запросом
7	`set interrupts off`; проверить команды, эхо и светофор	`OK`; статус P; обмен и индикация продолжаются
8	`set interrupts on`; повторить проверки	`OK`; статус I; обмен и индикация продолжаются
9	Пакет: off → `?` → on → `?` с CRLF	Последовательно: OK, статус P, OK, статус I; данные не очищаются при переключении
10	Неизвестная команда; timeout 0, −1, 1.5, abc, 2147484	Сообщение ERROR; прежние параметры сохраняются
11	CR, LF, CRLF; Backspace; строка длиннее 95 символов	Один ответ на завершённую команду; исправление ввода; ошибка длинной строки; следующая команда принимается
12	В обоих режимах вернуть timeout 4 и mode 1	Подтверждения OK; исходные настройки и поведение кнопки восстановлены

Таблица 6. План аппаратной проверки и ожидаемые результаты

8. Выводы

Реализовано управление светофором варианта 1 через UART на скорости 57600 бит/с: запрос состояния, включение и отключение обработки кнопки, изменение красного периода и переключение между опросом и прерываниями. Все команды разбираются в главном цикле; аппаратные особенности скрыты в драйверах.

Собственный драйвер USART6 использует кольцевые буферы, неблокирующий приём и передачу. Одна функция обслуживания применяется в двух режимах, а переключение I/P сохраняет очереди. Изучены роли флагов RXNE и TXE, работа разрешений прерываний USART и NVIC, а также необходимость коротких критических секций при совместном доступе к данным.

Анализ исходных файлов показывает соответствие реализованных механизмов заданию. Для подтверждения физических параметров обмена, точности длительностей и поведения при перегрузке необходимы результаты проверки на стенде; в предоставленных материалах они не зафиксированы.

9. Исходные коды собственной разработки

В листинги включены собственные модули UART и консоли, доработанный автомат и прикладная часть основной программы. Листинги встроены в этот документ текстом. Повторно не приводятся модули `button`, `sdk_board`, `custom_gpio`, разработанные в лабораторной работе № 1, а также HAL, CMSIS и код настройки тактирования, созданный на основе шаблона CubeMX. Это соответствует разрешению методички включать наиболее значимые части при большом объёме кода.

9.1. Прикладная часть основной программы: main.c

```
#include "main.h"

#include "button.h"
#include "console.h"
#include "sdk_board.h"
#include "traffic_light.h"
#include "uart_driver.h"

static Button button;
static TrafficLight traffic_light;
static Console console;

void SystemClock_Config(void);

static void set_traffic_lights(bool red, bool yellow, bool green)
{
    uint8_t mask = SDK_LED_NONE;
    mask |= red ? SDK_LED_RED : 0U;
    mask |= yellow ? SDK_LED_YELLOW : 0U;
    mask |= green ? SDK_LED_GREEN : 0U;
    sdk_board_write_leds(mask);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    sdk_board_init();
    const uint32_t now_ms = HAL_GetTick();
    button_init(&button, sdk_board_button_pressed(), now_ms, 30U);
    traffic_light_init(&traffic_light, now_ms, set_traffic_lights);
    if (!uart_init()) {
        Error_Handler();
    }
    console_init(&console);

    while (1) {
        uart_poll();
        const uint32_t current_ms = HAL_GetTick();
        traffic_light_update(&traffic_light, current_ms);
        const ButtonEvent event = button_update(&button, sdk_board_button_pressed(), current_ms);
        if (event.type == BUTTON_EVENT_PRESSED) {
            traffic_light_on_button_pressed(&traffic_light, current_ms);
        }
        console_update(&console, &traffic_light, current_ms);
    }
}
```

9.2. Интерфейс UART: uart_driver.h

```
#ifndef UART_DRIVER_H
#define UART_DRIVER_H

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

enum {
    UART_BAUD_RATE = 57600U,
    UART_RX_BUFFER_SIZE = 512U,
    UART_TX_BUFFER_SIZE = 2048U
};

bool uart_init(void);
/* Polling mode: one bounded RX/TX service pass. Interrupt mode: no-op. */
void uart_poll(void);
bool uart_read_byte(uint8_t *byte);
/* Enqueue the entire message or return false; never wait for transmission. */
bool uart_write(const char *data, size_t length);
size_t uart_tx_free(void);
void uart_set_interrupts(bool enabled);
bool uart_interrupts_enabled(void);
/* Report and clear a hardware RX error or a software RX buffer overflow. */
bool uart_take_rx_error(void);
void USART6_IRQHandler(void);

#endif
```

9.3. Драйвер USART6: опрос и прерывания: uart_driver.c

```
#include "uart_driver.h"

#include "custom_gpio.h"
#include "stm32f4xx_hal.h"

#define RX_MASK (UART_RX_BUFFER_SIZE - 1U)
#define TX_MASK (UART_TX_BUFFER_SIZE - 1U)
#define RX_ERRORS (USART_SR_ORE | USART_SR_NE | USART_SR_FE | USART_SR_PE)

_Static_assert((UART_RX_BUFFER_SIZE & RX_MASK) == 0U, "RX size must be a power of two");
_Static_assert((UART_TX_BUFFER_SIZE & TX_MASK) == 0U, "TX size must be a power of two");

static uint8_t rx_data[UART_RX_BUFFER_SIZE];
static uint8_t tx_data[UART_TX_BUFFER_SIZE];
static volatile uint32_t rx_head;
static volatile uint32_t rx_tail;
static volatile uint32_t tx_head;
static volatile uint32_t tx_tail;
static volatile bool rx_error;
static volatile bool use_interrupts;

static uint32_t lock(void)
{
    const uint32_t previous = __get_PRIMASK();
    __disable_irq();
    return previous;
}

static void unlock(uint32_t previous)
{
    __set_PRIMASK(previous);
}

/* Called only from the USART ISR or a short critical section in main. */
static void service_hardware(void)
{
    const uint32_t status = USART6->SR;
    if ((status & (USART_SR_RXNE | RX_ERRORS)) != 0U) {
        /* RM0090: SR read followed by DR read clears RXNE and RX errors. */
        const uint8_t byte = (uint8_t)USART6->DR;
        if ((status & RX_ERRORS) != 0U) {
            rx_error = true;
        }
        if ((status & USART_SR_RXNE) != 0U
            && (status & (USART_SR_NE | USART_SR_FE | USART_SR_PE)) == 0U) {
            const uint32_t next = (rx_head + 1U) & RX_MASK;
            if (next != rx_tail) {
                rx_data[rx_head] = byte;
                __DMB();
                rx_head = next;
            } else {
                rx_error = true;
            }
        }
    }

    if ((status & USART_SR_TXE) != 0U) {
        if (tx_tail != tx_head) {
            USART6->DR = tx_data[tx_tail];
            tx_tail = (tx_tail + 1U) & TX_MASK;
        }
        if (tx_tail == tx_head) {
            USART6->CR1 &= ~USART_CR1_TXEIE;
        }
    }
}

bool uart_init(void)
{
    NVIC_DisableIRQ(USART6_IRQn);
    RCC->APB2ENR |= RCC_APB2ENR_USART6EN;
    (void)RCC->APB2ENR;
    RCC->APB2RSTR |= RCC_APB2RSTR_USART6RST;
    RCC->APB2RSTR &= ~RCC_APB2RSTR_USART6RST;

    rx_head = rx_tail = tx_head = tx_tail = 0U;
    rx_error = false;
    use_interrupts = false;

    /* USART6 is on APB2, oversampling by 16, 8 data bits, no parity, 1 stop bit. */
    USART6->CR1 = 0U;
    USART6->CR2 = 0U;
    USART6->CR3 = 0U;
    USART6->BRR = (HAL_RCC_GetPCLK2Freq() + UART_BAUD_RATE / 2U) / UART_BAUD_RATE;
    USART6->CR1 = USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;

    const CustomGpioConfig tx_config = {
        .mode = CUSTOM_GPIO_MODE_ALTERNATE,
        .output_type = CUSTOM_GPIO_PUSH_PULL,
        .speed = CUSTOM_GPIO_SPEED_LOW,
        .pull = CUSTOM_GPIO_NO_PULL,
        .alternate_function = 8U,
        .output_high = true
    };
    CustomGpioConfig rx_config = tx_config;
    rx_config.pull = CUSTOM_GPIO_PULL_UP;
    if (!custom_gpio_init(GPIOC, (uint16_t)(1U << 6U), &tx_config)
        || !custom_gpio_init(GPIOC, (uint16_t)(1U << 7U), &rx_config)) {
        USART6->CR1 = 0U;
        return false;
```

```
    }

    NVIC_SetPriority(USART6_IRQn, 5U);
    NVIC_ClearPendingIRQ(USART6_IRQn);
    uart_set_interrupts(true);
    return true;
}

void uart_poll(void)
{
    if (!use_interrupts) {
        const uint32_t previous = lock();
        service_hardware();
        unlock(previous);
    }
}

bool uart_read_byte(uint8_t *byte)
{
    if (byte == NULL) {
        return false;
    }
    const uint32_t previous = lock();
    const bool available = rx_tail != rx_head;
    if (available) {
        *byte = rx_data[rx_tail];
        rx_tail = (rx_tail + 1U) & RX_MASK;
    }
    unlock(previous);
    return available;
}

size_t uart_tx_free(void)
{
    /* Only main produces TX data. ISR can only increase the free space. */
    return (size_t)((tx_tail - tx_head - 1U) & TX_MASK);
}

bool uart_write(const char *data, size_t length)
{
    if (data == NULL || length > uart_tx_free()) {
        return false;
    }
    uint32_t head = tx_head;
    for (size_t i = 0U; i < length; ++i) {
        tx_data[head] = (uint8_t)data[i];
        head = (head + 1U) & TX_MASK;
    }
    /* Publish only after copying. Keep interrupts enabled during the copy. */
    const uint32_t previous = lock();
    __DMB();
    tx_head = head;
    if (use_interrupts && tx_head != tx_tail) {
        USART6->CR1 |= USART_CR1_TXEIE;
    }
    unlock(previous);
    return true;
}

void uart_set_interrupts(bool enabled)
{
    const uint32_t previous = lock();
    NVIC_DisableIRQ(USART6_IRQn);
    USART6->CR1 &= ~(USART_CR1_RXNEIE | USART_CR1_TXEIE | USART_CR1_PEIE | USART_CR1_TCIE);
    USART6->CR3 &= ~USART_CR3_EIE;
    /* Preserve both rings and consume a possible pending RX byte before switching. */
    service_hardware();
    use_interrupts = enabled;
    NVIC_ClearPendingIRQ(USART6_IRQn);
    if (enabled) {
        USART6->CR3 |= USART_CR3_EIE;
        USART6->CR1 |= USART_CR1_RXNEIE;
        if (tx_tail != tx_head) {
            USART6->CR1 |= USART_CR1_TXEIE;
        }
        NVIC_EnableIRQ(USART6_IRQn);
    }
    unlock(previous);
}

bool uart_interrupts_enabled(void)
{
    return use_interrupts;
}

bool uart_take_rx_error(void)
{
    const uint32_t previous = lock();
    const bool error = rx_error;
    rx_error = false;
    unlock(previous);
    return error;
}

void USART6_IRQHandler(void)
{
    if (use_interrupts) {
        service_hardware();
    }
}
```

9.4. Структура консоли: console.h

```
#ifndef CONSOLE_H
#define CONSOLE_H

#include <stdbool.h>
#include <stddef.h>

#include "traffic_light.h"

typedef struct {
    char line[96];
    size_t length;
    bool discard_line;
    bool previous_cr;
    bool rx_failed;
} Console;

void console_init(Console *console);
void console_update(Console *console, TrafficLight *light, uint32_t now_ms);

#endif
```

9.5. Командный интерфейс: console.c

```
#include "console.h"

#include <stdio.h>
#include <string.h>

#include "uart_driver.h"

/* Enough space for echo, an error and the longest status response in one call. */
enum { CONSOLE_TX_RESERVE = 256U };

static void reply(const char *message)
{
    (void)uart_write("\r\n", 2U);
    (void)uart_write(message, strlen(message));
    (void)uart_write("\r\n", 2U);
}

static bool parse_seconds(const char *text, uint32_t *seconds)
{
    uint32_t value = 0U;
    if (*text == '\0') {
        return false;
    }
    while (*text != '\0') {
        if (*text < '0' || *text > '9') {
            return false;
        }
        const uint32_t digit = (uint32_t)(*text - '0');
        if (value > (TRAFFIC_MAX_TIMEOUT_SECONDS - digit) / 10U) {
            return false;
        }
        value = value * 10U + digit;
        ++text;
    }
    if (value == 0U) {
        return false;
    }
    *seconds = value;
    return true;
}

static void execute(char *line, TrafficLight *light, uint32_t now_ms)
{
    while (*line == ' ' || *line == '\t') {
        ++line;
    }
    size_t length = strlen(line);
    while (length > 0U && (line[length - 1U] == ' ' || line[length - 1U] == '\t')) {
        line[--length] = '\0';
    }
    if (length == 0U) {
        return;
    }

    if (strcmp(line, "?") == 0) {
        char status[128];
        (void)snprintf(status, sizeof status, "%s; mode %u; timeout %lu; %c",
            traffic_light_state_name(light), light->button_enabled ? 1U : 2U,
            (unsigned long)(light->red_ms / 1000U), uart_interrupts_enabled() ? 'I' : 'P');
        reply(status);
    } else if (strcmp(line, "set mode 1") == 0) {
        light->button_enabled = true;
        reply("OK");
    } else if (strcmp(line, "set mode 2") == 0) {
        light->button_enabled = false;
        reply("OK");
    } else if (strcmp(line, "set interrupts on") == 0) {
        uart_set_interrupts(true);
        reply("OK");
    } else if (strcmp(line, "set interrupts off") == 0) {
        uart_set_interrupts(false);
        reply("OK");
    } else if (strncmp(line, "set timeout ", 12U) == 0) {
```

```
        uint32_t seconds;
        if (parse_seconds(line + 12U, &seconds)
            && traffic_light_set_timeout(light, seconds, now_ms)) {
            reply("OK");
        } else {
            reply("ERROR: timeout must be an integer from 1 to 2147483 seconds");
        }
    } else {
        reply("ERROR: unknown command");
    }
}

void console_init(Console *console)
{
    console->length = 0U;
    console->discard_line = false;
    console->previous_cr = false;
    console->rx_failed = false;
    reply("SDK-1.1M lab 2: 57600 8N1; mode 1; timeout 4; I. Enter ? + Enter.");
}

void console_update(Console *console, TrafficLight *light, uint32_t now_ms)
{
    /* Leave RX queued until there is room for the echo and a full response. */
    if (uart_tx_free() < CONSOLE_TX_RESERVE) {
        return;
    }
    if (uart_take_rx_error()) {
        console->discard_line = true;
        console->rx_failed = true;
    }
    uint8_t byte;
    if (!uart_read_byte(&byte)) {
        return;
    }
    const char character = (char)byte;
    (void)uart_write(&character, 1U);

    const bool paired_lf = console->previous_cr && byte == '\n';
    console->previous_cr = byte == '\r';
    if (paired_lf) {
        return;
    }
    if (byte == '\r' || byte == '\n') {
        if (console->discard_line) {
            reply(console->rx_failed ? "ERROR: UART receive error; command discarded"
                                     : "ERROR: command too long or invalid character");
        } else {
            console->line[console->length] = '\0';
            execute(console->line, light, now_ms);
        }
        console->length = 0U;
        console->discard_line = false;
        console->rx_failed = false;
    } else if (byte == '\b' || byte == 127U) {
        if (!console->discard_line && console->length > 0U) {
            --console->length;
            (void)uart_write(" \b", 2U);
        }
    } else if (!console->discard_line) {
        if ((byte < 32U && byte != '\t') || byte > 126U
            || console->length >= sizeof console->line - 1U) {
            console->discard_line = true;
        } else {
            console->line[console->length++] = character;
        }
    }
}
```

9.6. Интерфейс доработанного автомата: traffic_light.h

```
#ifndef TRAFFIC_LIGHT_H
#define TRAFFIC_LIGHT_H

#include <stdbool.h>
#include <stdint.h>

enum {
    TRAFFIC_DEFAULT_RED_MS = 4000U,
    TRAFFIC_GREEN_MS = 1000U,
    TRAFFIC_BLINK_HALF_PERIOD_MS = 250U,
    TRAFFIC_BLINK_EDGE_COUNT = 8U,
    TRAFFIC_YELLOW_MS = 1000U,
    /* Keep every deadline interval below INT32_MAX milliseconds. */
    TRAFFIC_MAX_TIMEOUT_SECONDS = 2147483U
};

typedef enum {
    TRAFFIC_STATE_RED = 0,
    TRAFFIC_STATE_GREEN,
    TRAFFIC_STATE_BLINKING_GREEN,
    TRAFFIC_STATE_YELLOW
} TrafficLightState;

typedef void (*TrafficLightOutput)(bool red, bool yellow, bool green);

typedef struct {
    TrafficLightState state;
    uint32_t state_started_ms;
```

```
    uint32_t deadline_ms;
    uint32_t red_ms;
    uint8_t blink_edge;
    bool shorten_next_red;
    bool red_shortened;
    bool button_enabled;
    TrafficLightOutput output;
} TrafficLight;

void traffic_light_init(TrafficLight *light, uint32_t now_ms, TrafficLightOutput output);
void traffic_light_on_button_pressed(TrafficLight *light, uint32_t now_ms);
void traffic_light_update(TrafficLight *light, uint32_t now_ms);
bool traffic_light_set_timeout(TrafficLight *light, uint32_t seconds, uint32_t now_ms);
const char *traffic_light_state_name(const TrafficLight *light);

#endif
```

9.7. Автомат светофора и изменение тайм-аута: traffic_light.c

```
#include "traffic_light.h"

static bool deadline_reached(uint32_t now_ms, uint32_t deadline_ms)
{
    return (int32_t)(now_ms - deadline_ms) >= 0;
}

static void apply_output(const TrafficLight *light)
{
    const bool red = light->state == TRAFFIC_STATE_RED;
    const bool yellow = light->state == TRAFFIC_STATE_YELLOW;
    const bool green = light->state == TRAFFIC_STATE_GREEN
        || (light->state == TRAFFIC_STATE_BLINKING_GREEN && light->blink_edge % 2U == 0U);
    light->output(red, yellow, green);
}

static void enter_state(TrafficLight *light, TrafficLightState state, uint32_t now_ms)
{
    light->state = state;
    light->state_started_ms = now_ms;
    light->blink_edge = 0U;
    switch (state) {
        case TRAFFIC_STATE_RED:
            light->red_shortened = light->shorten_next_red;
            light->shorten_next_red = false;
            light->deadline_ms = now_ms + (light->red_shortened ? light->red_ms / 4U : light->red_ms);
            break;
        case TRAFFIC_STATE_GREEN:
            light->deadline_ms = now_ms + TRAFFIC_GREEN_MS;
            break;
        case TRAFFIC_STATE_BLINKING_GREEN:
            light->deadline_ms = now_ms + TRAFFIC_BLINK_HALF_PERIOD_MS;
            break;
        case TRAFFIC_STATE_YELLOW:
            light->deadline_ms = now_ms + TRAFFIC_YELLOW_MS;
            break;
        default:
            break;
    }
    apply_output(light);
}

void traffic_light_init(TrafficLight *light, uint32_t now_ms, TrafficLightOutput output)
{
    light->red_ms = TRAFFIC_DEFAULT_RED_MS;
    light->shorten_next_red = false;
    light->red_shortened = false;
    light->button_enabled = true;
    light->output = output;
    enter_state(light, TRAFFIC_STATE_RED, now_ms);
}

void traffic_light_on_button_pressed(TrafficLight *light, uint32_t now_ms)
{
    if (!light->button_enabled || light->state == TRAFFIC_STATE_GREEN) {
        return;
    }
    if (light->state == TRAFFIC_STATE_RED) {
        const uint32_t deadline = light->state_started_ms + light->red_ms / 4U;
        light->red_shortened = true;
        if (deadline_reached(now_ms, deadline)) {
            enter_state(light, TRAFFIC_STATE_GREEN, now_ms);
        } else {
            light->deadline_ms = deadline;
        }
    } else {
        light->shorten_next_red = true;
    }
}

void traffic_light_update(TrafficLight *light, uint32_t now_ms)
{
    for (uint32_t step = 0U; step < 16U; ++step) {
        if (!deadline_reached(now_ms, light->deadline_ms)) {
            return;
        }
        const uint32_t scheduled_ms = light->deadline_ms;
        switch (light->state) {
            case TRAFFIC_STATE_RED:
                enter_state(light, TRAFFIC_STATE_GREEN, scheduled_ms);
```

```
                break;
            case TRAFFIC_STATE_GREEN:
                enter_state(light, TRAFFIC_STATE_BLINKING_GREEN, scheduled_ms);
                break;
            case TRAFFIC_STATE_BLINKING_GREEN:
                ++light->blink_edge;
                if (light->blink_edge >= TRAFFIC_BLINK_EDGE_COUNT) {
                    enter_state(light, TRAFFIC_STATE_YELLOW, scheduled_ms);
                } else {
                    light->deadline_ms = scheduled_ms + TRAFFIC_BLINK_HALF_PERIOD_MS;
                    apply_output(light);
                }
                break;
            case TRAFFIC_STATE_YELLOW:
                enter_state(light, TRAFFIC_STATE_RED, scheduled_ms);
                break;
            default:
                enter_state(light, TRAFFIC_STATE_RED, now_ms);
                return;
        }
    }
}

bool traffic_light_set_timeout(TrafficLight *light, uint32_t seconds, uint32_t now_ms)
{
    if (seconds == 0U || seconds > TRAFFIC_MAX_TIMEOUT_SECONDS) {
        return false;
    }
    light->red_ms = seconds * 1000U;
    if (light->state == TRAFFIC_STATE_RED) {
        light->deadline_ms = light->state_started_ms
            + (light->red_shortened ? light->red_ms / 4U : light->red_ms);
        traffic_light_update(light, now_ms);
    }
    return true;
}

const char *traffic_light_state_name(const TrafficLight *light)
{
    switch (light->state) {
        case TRAFFIC_STATE_RED: return "red";
        case TRAFFIC_STATE_GREEN: return "green";
        case TRAFFIC_STATE_BLINKING_GREEN: return "blinking green";
        case TRAFFIC_STATE_YELLOW: return "yellow";
        default: return "unknown";
    }
}
```